Truth — System Architecture

Document: `architecture.md`
Status: Proposed architecture baseline
Version: 0.1
Purpose: Define the technology layers, system boundaries, data ownership, security model, execution model, and architectural invariants for Truth.

---

1. Architecture Overview

Truth is a modular, local-first personal intelligence platform. It runs initially as an application on an existing operating system. Its architecture separates user interfaces, API boundaries, the Truth Kernel, Runtime, Oracle, deterministic Compute Engine, shared data contracts, persistence, security, and external integrations.

The Kernel governs request processing and accepted state transitions. The Runtime coordinates component lifecycles and task execution. TIR defines shared data contracts. The Oracle interprets information and produces recommendations. The Compute Engine performs explicitly defined deterministic calculations. Persistence maintains events, current state, and memory records. Security and policy enforcement control access to data and executable capabilities.

Components must communicate through defined interfaces rather than directly modifying one another's internal state.

This document defines the intended architecture. It does not assert that every component, folder, dependency, or service already exists.

---

2. Stack and Technology Layers

The stack below is the proposed implementation baseline. Existing repository contents and dependencies must be inspected before introducing or replacing technologies.

Layer	Proposed technology	Responsibility
User interface	React, TypeScript	Render the user-facing application and administrative console
Styling and design system	Tailwind CSS	Shared design tokens, responsive layouts, themes, and component styling
Frontend routing	React Router	Route between application views and protected admin screens
Frontend server state	TanStack Query	Fetch, cache, invalidate, and refresh server data
Frontend forms	React Hook Form, Zod	Form state and client-side schema validation
Frontend testing	Vitest, React Testing Library, Playwright	Unit, component, and end-to-end testing
API layer	Python, FastAPI	Expose versioned HTTP endpoints and coordinate request handling
API schemas	Pydantic	Validate request and response structures at API boundaries
Application coordination	Truth Kernel	Govern request processing, state transitions, and accepted commits
Execution coordination	Truth Runtime	Manage registered components, tasks, events, timeouts, and lifecycle
Shared representation	TIR, using versioned Pydantic models initially	Define canonical data contracts between modules
Deterministic computation	Python modules with explicit calculation contracts	Perform calculations, rule evaluation, dependency resolution, and verification

Layer	Proposed technology	Responsibility
Oracle and reasoning	Provider-neutral Python interface; selected AI provider behind an adapter	Interpret context, reason, compare options, and prepare recommendations
Persistence	SQLite initially, with a repository abstraction	Store local structured records and transactional state
Database access	A database access layer; SQLAlchemy is an option if needed	Separate database queries from domain logic
File storage	Local filesystem through a controlled storage interface	Store attachments, exports, and other file-based artefacts
Cache	In-memory cache initially, only where justified	Store disposable performance data and derived results
Background work	Runtime-managed tasks initially	Execute bounded tasks, track status, and handle cancellation or failure
Authentication	Authentication interface with a deployment-appropriate implementation	Establish the authenticated principal
Authorisation	Server-side policy and permission service	Enforce access to resources and operations
Security auditing	Structured application audit records	Record privileged actions, policy decisions, and relevant outcomes
Observability	Python structured logging and health endpoints	Diagnose failures and observe system behaviour
Configuration	Environment-specific configuration validated at startup	Control deployment settings without hardcoding secrets
Packaging and delivery	Python packaging and documented frontend build tooling	Reproducible installation, development, and testing

2.1 Technology selection rules

- Inspect the repository before installing dependencies or restructuring it.
- Retain working technologies unless a documented requirement justifies changing them.
- Keep the backend and frontend independently testable.
- Use versioned contracts to connect modules and interfaces.
- Introduce additional infrastructure only when a measurable requirement justifies it.
- Do not add Redis, Celery, Kafka, Kubernetes, a vector database, or a separate microservice architecture merely because Truth may need them in the future.
- Record significant technology decisions and their trade-offs in Architecture Decision Records (ADRs).

---

3. System Boundaries and Folder Ownership

The following is the proposed logical repository structure. It should be reconciled with the actual source tree before files are created or moved.

Truth/

├── README.md

├── project-overview.md

├── architecture.md

├── docs/

│ ├── architecture/

│ │ ├── decisions/

│ │ └── diagrams/

│ ├── prd/

│ ├── engineering/

│ └── security/

├── backend/

```
│   ├── app/
│   │   ├── api/
│   │   ├── kernel/
│   │   ├── runtime/
│   │   ├── tir/
│   │   ├── oracle/
│   │   ├── compute/
│   │   ├── registry/
│   │   ├── persistence/
│   │   ├── security/
│   │   ├── integrations/
│   │   ├── tasks/
│   │   ├── observability/
│   │   ├── config/
│   │   └── main.py
│   ├── tests/
│   │   ├── unit/
│   │   ├── integration/
│   │   └── security/
│   └── pyproject.toml
├── frontend/
```

```
│   ├── src/
│   │   ├── app/
│   │   ├── components/
│   │   ├── features/
│   │   ├── services/
│   │   ├── hooks/
│   │   ├── types/
│   │   └── styles/
│   └── package.json
└── .gitignore
```

This is an architectural proposal, not a requirement to create every directory immediately. The coding agent must preserve useful existing work and implement only the folders needed for the current milestone.

3.1 Backend ownership

Folder	Owns	Must not own
`api/`	HTTP routes, request parsing, response mapping, API versioning	Core business rules or direct unrestricted database writes
`kernel/`	Request lifecycle, state-transition coordination, commit decisions	Provider-specific AI logic or direct UI concerns
`runtime/`	Component registry, dependency initialisation, dispatch, execution lifecycle, shutdown	Domain-specific decisions that belong to the Kernel or domain modules
`tir/`	Canonical schemas, validation contracts, version	Unrestricted external execution

Folder	Owns	Must not own
	compatibility, transformation types	
`oracle/`	Context assembly, reasoning orchestration, evidence and inference classification, recommendations	Independent permission grants or uncontrolled state mutation
`compute/`	Deterministic calculations, calculation registry, dependency evaluation, result verification	Treating AI-generated numerical prose as authoritative computation
`registry/`	Entity and relationship records, operational dependency graph, traversal, impact analysis	Assuming every relationship is a verified fact
`persistence/`	Database access, repositories, transactions, migrations, event and state storage	Owning business decisions about whether a state transition is allowed
`security/`	Authentication integration, policy evaluation, capability checks, authorisation, audit rules	Trusting client-side access controls as the security boundary
`integrations/`	Explicit adapters for external applications, APIs, and services	Bypassing Runtime and policy enforcement
`tasks/`	Task definitions, task-specific handlers, status and outcome contracts	Unregistered execution or silent privilege escalation
`observability/`	Structured logs, health checks, metrics interfaces, correlation	Logging secrets or unnecessary private content

Folder	Owns	Must not own
`config/`	Typed settings, configuration loading, startup validation	Hardcoded credentials or uncontrolled runtime configuration changes
`tests/`	Unit, integration, contract, and security tests	Depending on live external services when deterministic mocks are sufficient

3.2 Frontend ownership

Folder	Owns
`app/`	Application bootstrap, top-level routes, layout, and global providers
`components/`	Reusable interface components and shared design-system elements
`features/`	Feature-specific views and UI logic for Oracle, goals, activity, and administration
`services/`	Typed API clients and request/response mapping
`hooks/`	Reusable frontend hooks
`types/`	Frontend types derived from or aligned with backend API contracts
`styles/`	Global styles, theme variables, and design tokens

The frontend may hide or disable unavailable actions for usability, but it must never be the only place where access permissions are enforced.

3.3 Dependency direction

- API routes call application services and the Kernel.
- The Kernel coordinates TIR validation, retrieval, reasoning, policy checks, and accepted state transitions.
- The Runtime executes registered components and tasks through defined contracts.

- Domain modules use declared interfaces to access persistence, security, and external adapters.
- The persistence implementation sits behind a stable boundary so storage technology can change without rewriting domain logic.
- External integrations return structured results to the application; they do not bypass the Kernel's state-management rules.
- Avoid circular imports and circular module dependencies. If two components require one another, extract a shared contract or redesign the interface.

---

4. Storage Model

Truth must distinguish authoritative records from derived data and temporary performance data. A cache is not the source of truth, and a file is not automatically a database record.

4.1 Database

- **Initial proposal:** SQLite for a single-instance local deployment.
- The database stores structured, queryable records requiring durable persistence, relationships, or transactional updates.

Record category	**Examples**	**Storage rules**
Events	Accepted observations, state-change events, task outcome events	Preserve event identity, timestamps, source, and provenance
Current state	Current accepted state and versioned snapshots	Update through Kernel-controlled transactions
Memory records	Approved user context, retained observations, summaries with provenance	Apply privacy, sensitivity, retention, and correction rules
Goals and tasks	Goal definitions, success criteria, task status, dependencies	Store durable state and progress evidence references
TIR objects	Persisted entities, relationships, commands, results, and transformation records	Validate schemas and record schema versions

Record category	Examples	Storage rules
Dependency registry	Operational prerequisites, dependency type, version, and status	Preserve dependency direction and provenance
Permissions	Role assignments, grants, scopes, expiry, and policy metadata where applicable	Apply server-side access controls and audit important changes
Audit records	Privileged actions, policy decisions, security events, recovery actions	Use controlled retention and access policies
Integration metadata	Registered adapter IDs, non-secret configuration references, connection status	Keep credentials outside ordinary records

- Database writes involving related authoritative records should use transactions where required to prevent partially committed state.
- For future multi-user deployments, access-controlled records must be associated with an appropriate owner or tenant where relevant. Ownership must be enforced in database access paths and application services, not inferred from a user-supplied ID.

4.2 File storage

Use a controlled file-storage interface for data that is naturally file-based or too large to embed directly in ordinary records.

Examples:

- User-provided documents and attachments.
- Generated exports and reports.
- Large evidence artefacts.
- Imported source files.
- Optional local model files or other large assets, if later supported.

Rules:

- Store file metadata and access-control references in the database.
- Store file bytes through a defined storage interface.
- Do not treat an arbitrary file path from a request as trusted.
- Validate file type, size, and permitted location.

- Use opaque storage identifiers rather than exposing unrestricted filesystem paths.
- Apply ownership, access, deletion, and retention rules to the file as well as its metadata.
- Keep private user files out of version control.
- Do not store credentials in ordinary uploaded files or generated artefacts.

4.3 Cache

A cache may contain:

- Rebuildable retrieval indexes.
- Repeated query results with a defined freshness policy.
- Temporary context assembled for a reasoning request.
- Non-authoritative component health snapshots.
- Expensive derived results whose inputs and versions can be checked.

Rules:

- Every cached value must have a defined invalidation or expiry strategy.
- Cache keys must account for relevant user or tenant boundaries and permission context.
- A cache hit must not bypass authorisation.
- Derived calculation results must be invalidated when relevant inputs, rules, or versions change.
- Clearing the cache must not destroy authoritative user records or accepted system state.
- Sensitive content must not enter a shared cache without an explicit isolation policy.

4.4 Secrets and configuration

- Credentials, API keys, session secrets, and signing keys must not be stored as ordinary application data.
- Use environment-based configuration or an appropriate operating-system credential store for development and deployment.
- Use a platform-specific secure storage mechanism when available for local credentials.
- Do not commit `.env` files containing secrets.
- Validate required configuration at startup.
- Redact credentials and sensitive tokens from logs and error messages.
- Separate secret values from database records and ordinary configuration exports.

4.5 Data lifecycle

- The system must define how records are created, read, updated, corrected, exported, retained, and deleted.
- Historical events should not be silently overwritten to conceal a previous outcome. Corrections should be explicit and linked to the relevant prior record. Where a user

requests deletion, apply the approved privacy and retention policy to authoritative records, associated files, indexes, and derived copies.
- Audit retention requirements must be explicit; immutability must not become an excuse to retain personal data indefinitely without a defined purpose or policy.

---

5. Authentication and Access Model

Authentication establishes who or what is making a request. Authorisation determines which data and operations that principal may access. These are separate responsibilities.

5.1 Initial deployment model

- The initial deployment should support a controlled local user experience while keeping identity and permission logic behind interfaces that can support additional users later.
- The exact authentication mechanism must be selected after inspecting the existing repository and intended deployment model.
- For a private local prototype, use an explicit local-session or development authentication mechanism with clearly documented limitations.
- For a network-accessible deployment, use a properly configured authentication system and secure session handling.
- For a future multi-user deployment, integrate an appropriate identity provider or implement a separately reviewed authentication system.
- Administrative access must require an explicitly authorised principal; it must not be granted to every authenticated user.
- Mock authentication is permitted for isolated development and testing, but must not be mistaken for production security.

5.2 Principal and ownership

- Every protected operation must execute in the context of a known principal.
- A principal may represent a user or an authorised system component. Its identity and permissions must come from trusted authentication and policy context, not from unverified request-body fields.
- For user-owned resources:
    1. The server establishes the authenticated principal.
    2. The application resolves the resource using its identifier.
    3. The application verifies that the principal is allowed to access that resource.
    4. The operation is performed only after applicable policy checks pass.
    5. The outcome and relevant audit information are recorded.
- A user must not gain access to another user's data by changing an ID in a URL, query parameter, or request body.

- Future multi-user records should include an explicit ownership or tenant association wherever appropriate. Background jobs must preserve the initiating principal or an explicitly defined service identity, together with the applicable permission scope.

5.3 Roles and permissions

The administrative interface may define roles such as:

Role	Intended authority
Super Admin	Broad system administration, with sensitive actions explicitly audited
Operations Admin	Health monitoring, task inspection, and permitted operational controls
Security Admin	Security events, access policy administration, and approved containment actions
Support Admin	Limited user support and troubleshooting within defined scopes
Read-only Auditor	Read-only access to permitted audit and governance records

These roles are proposed defaults, not verified existing accounts or grants. The actual permission matrix must be defined before privileged operations are implemented. Roles group permissions, but sensitive actions should also be checked against resource scope, current policy, and required approval.

5.4 Capabilities, grants, and approvals

Truth must distinguish:

- **Capability:** what a registered component is technically able to do.
- **Permission grant:** what a principal is authorised to do, within a defined scope.
- **Approval:** an explicit decision required before a particular operation proceeds.
- **Policy:** the rules determining whether the operation is allowed under current conditions.

A registered capability does not automatically authorise its use. The policy check must occur at the execution boundary, immediately before the protected operation. High-risk operations must require appropriate approval or be denied.

5.5 Session and request security

- Protect session tokens and credentials.
- Apply secure cookie or token practices appropriate to the chosen authentication mechanism.
- Validate request bodies, parameters, and resource identifiers.
- Apply rate and resource limits where exposed interfaces require them.
- Prevent cross-user data access.
- Avoid returning private records or sensitive internal details to unauthorised principals.
- Audit privileged operations without unnecessarily logging private content.
- Fail closed when a required permission decision cannot be established.

---

6. AI and Reasoning Model

Truth should use a provider-neutral reasoning interface. The rest of the application must not depend directly on one vendor's model-specific API.

6.1 Oracle responsibilities

The Oracle may:

- Interpret a user's request.
- Retrieve relevant context through approved services.
- Distinguish evidence, assumptions, inferences, hypotheses, and unknowns.
- Compare options and identify trade-offs.
- Produce a proposed plan or structured recommendation.
- Request deterministic calculations from the Compute Engine.
- Prepare an operation request for policy evaluation and possible execution.

The Oracle must not directly write arbitrary database records, grant permissions, execute unregistered tools, or treat its own generated claims as verified external facts.

6.2 Reasoning request lifecycle

1. Validate the incoming request.
2. Retrieve only context the current principal is permitted to access.
3. Assemble a bounded reasoning context.
4. Invoke the configured reasoning provider through the Oracle adapter.
5. Validate the returned structure.
6. Record relevant evidence references, assumptions, uncertainty, and provider metadata where appropriate.

7. Send proposed calculations to the Compute Engine.
8. Send proposed operations to the policy and Runtime boundary.
9. Present the result and distinguish recommendations from verified outcomes.

Model outputs, retrieved documents, and external content must be treated as untrusted input. Instructions found inside retrieved content must not override system policy or grant execution authority.

6.3 Provider interface

The provider interface should define:

- Request and response schemas.
- Supported reasoning modes.
- Timeout and cancellation behaviour.
- Error mapping.
- Resource and usage limits where available.
- Data-sharing constraints.
- Model and provider metadata where available.
- Validation requirements for structured outputs.

Provider-specific request formats must remain inside the adapter.

6.4 Deterministic computation

- The Compute Engine must handle calculations whose correctness depends on explicit rules and reproducible inputs.
- AI may help formulate a calculation request, but the selected calculation must be validated and executed through a registered deterministic procedure.
- Results should record the calculation identifier, procedure version, validated inputs or references, and output. Monetary values must use an appropriate decimal representation and explicit rounding rules rather than relying on binary floating-point arithmetic where exact decimal arithmetic is required.

---

7. Background Tasks and Event Processing

Truth needs background execution for operations that are long-running, scheduled, retryable, or independent of the immediate response. This does not justify introducing a distributed queue in the first release.

7.1 Initial execution model

- Begin with Runtime-managed tasks inside the supported application process.
- Use this model for bounded operations such as memory indexing, dependency analysis, longer calculations, approved integrations, exports, and other supported background work.
- Each task must have a stable identifier and an explicit lifecycle: `RECEIVED` → `VALIDATED` → `QUEUED` → `RUNNING` → `SUCCEEDED`
- Alternative terminal or waiting states include `REJECTED`, `FAILED`, `CANCELLED`, `TIMED_OUT`, and `WAITING_FOR_APPROVAL`. Not every task must pass through every state. Transitions must be validated against the task lifecycle rules.

7.2 Durable task tracking

Persist task status when it must survive process restarts or be visible across sessions. A task record should include:

- Task ID and task type.
- Schema or handler version.
- Initiating principal or defined service identity.
- Input reference or validated payload.
- Correlation ID.
- Creation and update timestamps.
- Status and attempt count.
- Required permissions and approval references.
- Result reference or structured error.
- Relevant provenance.

7.3 Retries and idempotency

- Retries must be bounded and applied only where appropriate.
- An operation that may have external side effects must use an idempotency mechanism where the external service supports it, or another documented duplicate-protection strategy.
- A timeout does not prove that an external operation failed. If the outcome is unknown, the task must be marked accordingly and reconciled before an unsafe retry.
- Do not claim exactly-once execution unless the complete mechanism and its guarantees have been demonstrated.

7.4 Future queue infrastructure

- A durable queue or external worker system may be introduced when measured requirements demand independent worker processes, persistent scheduling, throughput, or reliable cross-process coordination.
- Any such change must preserve the Runtime task contract, authorisation boundary, task status model, idempotency rules, and observability.

---

8. API and Module Contracts

- The initial API should use HTTP with JSON request and response bodies unless a specific capability justifies another protocol.
- Version public API routes, for example `/api/v1/`.
- Validate requests at the boundary.
- Return structured errors with stable machine-readable codes.
- Keep internal exceptions and secrets out of client-facing errors.
- Use explicit response schemas rather than returning database models directly.
- Define pagination, filtering, and ordering rules for list endpoints.
- Document authentication requirements for protected endpoints.
- Assign correlation IDs for tracing requests and background work.
- Avoid exposing internal filesystem paths, database structure, provider credentials, or privileged implementation details.
- Internal modules should expose narrow, documented interfaces. A module must not import another module's private implementation merely to bypass its public contract.

---

9. Observability, Failure Handling, and Recovery

Truth must make failures observable without leaking sensitive information. The system should provide:

- Structured logs with severity, timestamp, component, and correlation ID.
- Health checks that distinguish process availability from dependency health.
- Explicit task and component status.
- Structured validation, policy, dependency, and execution errors.
- A record of significant state transitions and privileged operations.
- Startup checks for required configuration and persistence availability.
- A documented backup and restore process for supported data.
- Recovery verification before restoring normal operation.

A failed operation must not be silently represented as successful. A partial operation must report which steps completed and which remain unresolved. Recovery procedures must preserve relevant evidence, validate restored data, re-establish the required permissions and configuration, and verify system health before normal operation resumes.

---

10. Architectural Invariants

These rules must never be violated by application code, AI-generated changes, integrations, or background workers.

1. **All protected operations require server-side authorisation.** Hiding a button or route in the frontend is not sufficient. A caller cannot gain access by changing a resource identifier or constructing a direct request.
2. **The Oracle cannot authorise itself.** AI recommendations and generated tool requests are proposals, not permission grants. Every protected operation must pass through the applicable policy and execution boundary.
3. **The Kernel governs accepted state transitions.** Modules may return results or propose changes, but must not bypass the defined state-management boundary to mutate authoritative state.
4. **TIR contracts must be validated and versioned.** Components must not silently exchange incompatible payloads or assume that an unknown schema is valid.
5. **Deterministic calculations must use explicit procedures.** Numerical or rule-based results must not be accepted merely because an AI model states them confidently.
6. **Evidence and inference must remain distinguishable.** User reports, external claims, verified results, generated interpretations, hypotheses, and unknowns must not be silently merged into one undifferentiated fact record.
7. **Ownership must be checked on every protected resource access.** User-controlled IDs are not proof of ownership. Background tasks must preserve a valid initiating principal or explicitly scoped service identity.
8. **Capabilities and permissions are separate.** A component's technical ability to perform an operation never grants permission to perform it.
9. **Untrusted content must not become executable authority.** User inputs, retrieved documents, external responses, and model outputs cannot override trusted policy, register privileged handlers, or grant themselves permissions.
10. **Committed data must not be silently corrupted or discarded.** Related state changes must use appropriate transaction boundaries. Corrections and supersession must be explicit, and restart recovery must be tested.
11. **Caches are never authoritative storage.** Clearing a cache must not destroy committed state, and cached data must never bypass current authorisation or relevant invalidation rules.

12. **External side effects must be tracked honestly.** A timeout or lost response must not automatically be interpreted as proof of failure. Retried operations must use appropriate duplicate-protection mechanisms.
13. **Secrets must not be committed to source control or exposed in ordinary logs.** Credentials and tokens must be handled through approved configuration or secure storage mechanisms.
14. **No component may bypass Runtime and policy enforcement for protected execution.** Integrations, scheduled jobs, and administrative actions must use the same applicable security boundaries as ordinary requests.
15. **Failure must not be disguised as success.** Task status, response status, and committed state must reflect the verified outcome, including partial or uncertain outcomes.
16. **Persistence changes must preserve defined ownership and privacy rules.** Storage refactoring must not silently expose private records, remove required provenance, or evade approved deletion and retention policies.
17. **Module dependencies must remain explicit and acyclic where required.** New dependencies must be declared and validated; circular dependencies must not be hidden through import tricks or shared mutable global state.
18. **Local-first behaviour must not be overstated.** A feature may be described as offline-capable only when its required compute, data, storage, and execution path have been tested without unavailable remote services.
19. **New integrations must use explicit contracts.** External adapters must validate inputs and outputs, declare side effects and permissions, and return structured failures.
20. **Tests and implementation status must be reported honestly.** Generated code, mocked responses, and unexecuted tests do not prove that a feature works. A task is complete only when its acceptance criteria have been verified.

---

11. Testing Requirements

Every architectural layer must have tests appropriate to its responsibility.

Test type	**Required coverage**
Unit tests	Kernel transitions, calculations, validators, policy rules, dependency graph operations
Contract tests	TIR schemas, API request and response structures, component interfaces, adapter contracts

Test type	Required coverage
Integration tests	Database transactions, persistence recovery, Runtime dispatch, Kernel-to-module flow
Security tests	Ownership checks, unauthorised operations, privilege escalation attempts, untrusted inputs
Failure tests	Timeouts, missing dependencies, invalid provider responses, partial outcomes, interrupted tasks
Frontend tests	Forms, access-denied states, task status, error handling, key user journeys
End-to-end tests	A complete supported user request through the interface, backend, persistence, and returned result

Tests should use controlled fixtures and mocks where appropriate. Live external services must not be required for routine deterministic tests.

---

12. Implementation Boundaries

Implement the architecture in small, verifiable increments.

Initial baseline:

1. Audit the repository and verify the actual stack.
2. Establish shared schemas, configuration, and structured errors.
3. Implement a minimal Kernel and Runtime request path.
4. Add transactional persistence and state-transition tests.
5. Introduce TIR validation and one deterministic calculation.
6. Add basic ownership and policy enforcement before protected operations.
7. Build the smallest usable frontend journey against actual implemented endpoints.
8. Expand the registry, Oracle, admin console, and integrations through bounded tasks.

Do not implement all proposed directories, technologies, and future capabilities at once. Each new component must have a clear owner, interface, dependency contract, and acceptance test.

---

13. Open Architecture Decisions

The following decisions must be verified or formally approved before they are treated as fixed implementation facts.

Decision	Current position	Required resolution
Existing backend stack	README refers to Python, FastAPI, Uvicorn, and pytest	Inspect actual files, dependencies, and tests
Frontend stack	React, TypeScript, Tailwind CSS proposed	Confirm against the repository and delivery needs
Database	SQLite proposed for the initial local deployment	Verify transaction, migration, and concurrency requirements
Database access library	Not confirmed; SQLAlchemy is an option	Select after inspecting existing persistence code
Authentication mechanism	Deployment-dependent interface proposed	Define the first deployment environment and trust boundary
Initial user model	Controlled local user experience	Decide whether the first deployed version is single-user or multi-user
AI provider	Provider-neutral adapter required	Select a provider or deterministic stub for the initial milestone
Background execution	Runtime-managed tasks initially	Define which tasks need durable execution across restarts
File storage	Local storage interface proposed	Define supported file types, limits, access rules, and retention
Frontend and backend packaging	Separate logical applications proposed	Verify whether the repository should use one root project or a monorepo layout

Until resolved, these are explicit assumptions rather than verified repository facts.

---

14. Definition of Done

An architectural component is complete only when:

- Its responsibility and ownership boundary are documented.
- Its inputs, outputs, dependencies, and failure modes are defined.
- Its data access and permission requirements are explicit.
- Its implementation follows the agreed contract.
- Relevant automated tests have been run and their actual results reported.
- Logs and errors are sufficient to diagnose expected failures.
- Its documentation matches the implementation.
- No architectural invariant has been violated.

This architecture is successful when Truth can grow by adding well-defined, independently testable capabilities without losing control of state, permissions, data ownership, execution, or recovery.